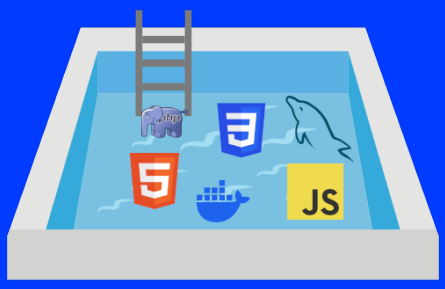
{EPITECH}

BOOTCAMP WEB_

< DAY 06 - JAVASCRIPT BASICS />



BOOTCAMP WEB

The next few days will be dedicated to the JavaScript language. Let's get comfy with it.



{EPITECH}

How to test

Today we will ask you to write functions in JavaScript. In JavaScript, your files are called **modules**.
By default, each function in a module can only be called from inside the module.

But to be able to use it from another module (a test file for instance) you have to **export** it.

Here's an `example.mjs` file:

```
function example() {
  console.log("Hello World!");
}
```

To be able to call this function from another file, you have to **export** it like this:

```
export function example() {
  console.log("Hello World!");
}
```

Then in another file, let's say `test.mjs`, you can **import** it and use it.

```
import { example } from "./example.mjs";

example();
```

But how do you run it?
To run your JS file, we will use Node.js.
On the download page, select the LTS version for your operating system using `nvm` with `npm` then follow the instructions.



If you're on Windows, we suggest you use **WSL** and follow the instructions for *Linux*.

Then run it using this command:

```
node test.mjs
Hello World!
```


{EPITECH}

Task00 — Quick warm-up

Before you start, make sure you can run a simple file.

Create a file named `task00.mjs`:

```
export function sayHello(name) {
  console.log("Hello " + name + "!");
}
```

Then run:

```
node --input-type=module -e "import('./task00.mjs').then(m => m.sayHello('Epitech'))"
Hello Epitech!
```

If this works, you're ready to start coding!



Task01

Delivery: `./task01.mjs`
Prototype: `drawTriangle(height)`

Write a function `drawTriangle` that:

- ✓ takes a triangle height as parameter;
- ✓ draws a triangle on the standard output;
- ✓ is exported and contained in a `task01.mjs` file.

```
cat task01.mjs
export function drawTriangle(height) {
  // your code here
}
```

Your function will be tested the following way:

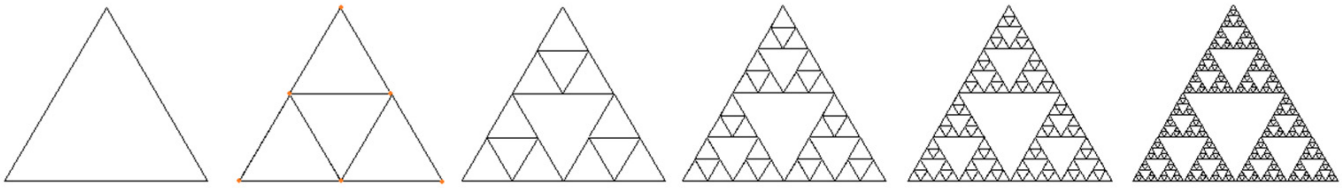
```
cat task01_tester.mjs
import { drawTriangle } from './task01.mjs';

drawTriangle(5);
```

```
node task01_tester.mjs
$
$$
$$$
$$$$
$$$$$
```



You can use a loop or the `repeat()` method to print `$` characters.



{EPITECH}

Task02

Delivery: `./task02.mjs`
Prototype: `arraysAreEqual(arr1, arr2)`

Write a function `arraysAreEqual` that:

- ✓ takes two arrays as arguments;
- ✓ returns `true` if both arrays contain the same elements in the same order;
- ✓ returns `false` otherwise;
- ✓ is exported and contained in a `task02.mjs` file.

```
cat task02.mjs
export function arraysAreEqual(arr1, arr2) {
  // your code here
}
```

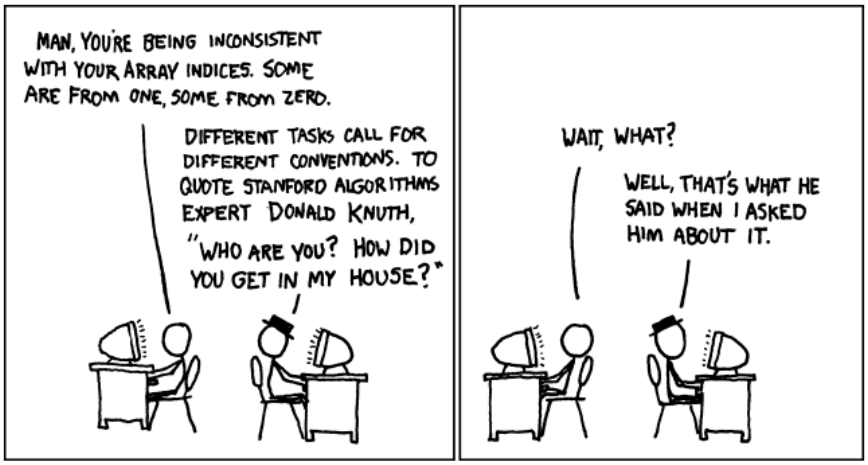
Your function will be tested the following way (testing values may vary):

```
cat task02_tester.mjs
import { arraysAreEqual } from './task02.mjs';
console.log(arraysAreEqual([1, 2], [1, 4]) ? 'True' : 'False');
```

```
node task02_tester.mjs
False
```



Two arrays are equal if they have the same length and each element at the same index is equal.



Task03

Delivery: `./task03.mjs`
Prototype: `countGs(str)`

Write a `countGs` function that:

- ✓ takes a string as parameter;
- ✓ returns the number of `'G'` it contains;
- ✓ is exported and contained in a `task03.mjs` file.

```
import { countGs } from './task03.mjs';
console.log(countGs("Goggles"));
```

```
node example.mjs
3
```



Task04

Delivery: `./task04.mjs`
Prototype: `fizzBuzz(num)`

Write a `fizzBuzz` function that:

- ✓ takes a number as parameter;
- ✓ prints all the numbers from 1 to this number;
- ✓ is exported and contained in a `task04.mjs` file;
- ✓ complies with the following requirements:
  1. print ”Fizz” instead of the number if it is divisible by 3;
  2. print ”Buzz” instead of the number if it is divisible by 5;
  3. print ”FizzBuzz” instead of the number if it is divisible by both 3 and 5.



The output terms (be it a number or a string) should be **comma + space separated**, for example:

```
1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz, 11, Fizz, 13, 14, FizzBuzz
```



{EPITECH}

Task05

Delivery: `./task05.mjs`
Prototype: `range(start, end, step)`

Write a `range` function that:

- ✓ takes 3 integers named `start`, `end` and `step` as arguments;
- ✓ returns an array containing all the numbers from `start` up to `end` (inclusive);
- ✓ with an increment corresponding to the optional third argument `step`;
- ✓ is exported and contained in a `task05.mjs` file.

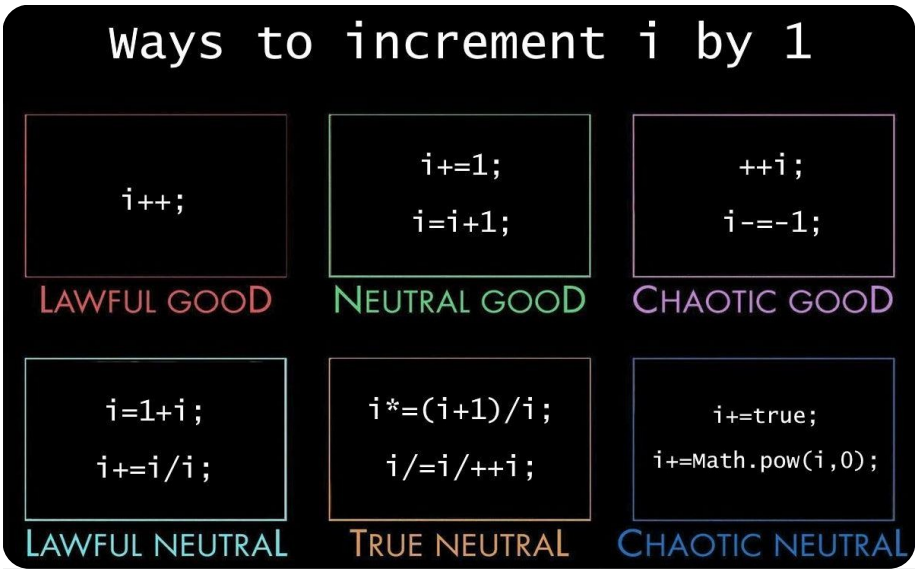


If `step` is not provided, the default increment should be **1** if `start < end`, and **-1** otherwise.
Your code should correctly handle both increasing and decreasing ranges.

```
import { range } from './task05.mjs';

console.log(range(1, 10, 2));
console.log(range(19, 22));
console.log(range(5, 2, -1));
```

```
node example.mjs
[ 1, 3, 5, 7, 9 ]
[ 19, 20, 21, 22 ]
[ 5, 4, 3, 2 ]
```



Task06

Delivery: `./task06.mjs`
Prototype: `arrayFiltering(array, test)`

In `task06.mjs`, write the `arrayFiltering` function that:

- ✓ takes two arguments: `array` and `test` (where `test` is a function returning a boolean);
- ✓ calls the `test` function for each element contained in `array`;
- ✓ returns a new array, containing the values for which `test` returned true.

```
cat example.mjs
// ...
const toFilter = [1, 2, 3, 4, 5, 6, 7, 8, 9];
const res = arrayFiltering(toFilter, function (value) {
  return value % 3 === 0;
});
console.log(res);
```

```
node example.mjs
[3, 6, 9]
```



Your function should NOT care about the implementation of the `test` function.

{EPITECH}

Task07

Delivery: `./task07.mjs`
Prototype: `objectsDeeplyEqual(cmp1, cmp2)`

In `task07.mjs`, write the function `objectsDeeplyEqual` that takes two arguments and returns `true` only if:

- ✓ the arguments have the same value;
- ✓ or they are objects with the same properties whose values are equal when compared with a **recursive** call to `objectsDeeplyEqual`.

```
cat example.mjs
// ...
const obj = {here: {is: "an"}, object: 2};
console.log(objectsDeeplyEqual(obj, obj));
console.log(objectsDeeplyEqual(obj, {here: 1, object: 2}));
console.log(objectsDeeplyEqual(obj, {here: {is: "an"}, object: 2}));
```

```
node example.mjs
true
false
true
```



Your function should figure out whether to compare two things by identity or by looking at their properties.
It doesn't have to cover every edge case — keep it simple, this is meant as a **challenge**.



Remember: `null` is also an ”object” in JavaScript.



Non-zero value



0



null



undefined

{EPITECH}

Need a break?

If you need a little break in your day or later, we strongly suggest you to:

- ✓ play the cyberpunk text-based incremental RPG bitburner;
- ✓ collect coins in the platform game JSRobot;
- ✓ code your way through the dungeons of warriorjs;
- ✓ program the elevators' movements in Elevator saga;
- ✓ solve the mazes of Untrusted, the katas of jskatas, or the puzzles of JSDares.



v 4.4

